基础算法 46 题·我自己的复习笔记

> 这份不是题解，也不是什么「套路大全」。
> 就是把做过的 46 道题重新翻一遍，把当时卡住的地方、后来才想明白的地方，用大白话记下来。
> 代码都取自 `002基础算法46题/.cpp带注释版本/`，是我自己写过的那些写法，可以打开对着看。
> 怎么用：做完一章，回来看看「要记得」那几条；哪条想不起来了，就翻回那道题的 `.cpp`。
> 看不懂的地方不用急，先往下看，回头再读一遍就顺了。

〇、先说三件最要紧的

回头翻这 46 道题，出现次数最多的就这三条：

1. **看到「求和」「乘积」「坐标到 1e9」，先把 `long long` 写上。** P8218 的前缀和、P1873 砍树累加、P1908 的逆序对个数、P1496 的坐标差，都是在这里翻的车。逆序对最多有 `n(n-1)/2`，`n=5e5` 时约 `1.25e11`，`int` 根本装不下。
2. **下标从 0 还是从 1，动笔前定死。** `cin` 读进来是 1~n，就全程 `1..n`、循环写 `i<=n`，别写着写着混了。P2004 我把行列转置存了，输出还得反着写 `maxY+1 maxX+1`，自己都差点绕晕。
3. **样例过了不等于对。** 自己再造几组：`n=1`、全是 0、答案全是负数、刚好卡在边界上。P1085 只有第 1 天超过 8 小时这种情况，样例里没有，`ans` 初值给 0 就会输出 0（应该是 1）。

顺带把《001时间复杂度》里那张表抄一份过来，省得再翻：

N 多大	大概能写	别写什么
≤ 20	全排列、子集枚举	—
≤ 300	三重循环	—
≤ 2000	双重循环	三重
≤ 1e5	一重、排序、二分	双重循环（1e10 次，跑不完）
≤ 1e8	一重扫描	排序都勉强
≥ 1e9	公式、√N、log N	跟 N 有关的循环

> **要记得**：`N ≤ 1e5` 还写双重循环，不是「慢一点」，是根本交不上去。P1873、P2440、P1638、P3029、P1908 都卡在这条线上。

一、枚举（01~05）

【枚举】说白了就是：想不出巧办法，那就一个一个试。前提是给一个候选，能马上判断它行不行，而且候选不能太多。

题号	题目	我记的一句
01	P1179 数字统计	从 1 数到 n，每个数用 `t%10`、`t/10` 把位拆开，数里面有几个 2
02	P2637 拍卖	出价**降序排好**再试售价：定在第 i 个人的出价上，前面 i+1 个人都买得起，收入就是 `(i+1)*p[i]`，内层循环直接不用写
03	P1085 不高兴的津津	打擂台找最大的 `a+b`，顺便记下是第几天
04	P6437 JACK	三层循环选三个数，`j=i+1`、`t=j+1`，和不超过 m 就更新答案
05	P8834 序列	二重循环比乘积

P2637 最值得回头看。它其实已经不算纯暴力了，是**排序帮我省掉了一层循环**：

```
sort(p, p + m, greater<int>());      // 出价从高到低
int limit = min(n, m);               // 草料只有 n 批，人再多也没用
for (int i = 0; i < limit; i++) {
    int sum = (i + 1) * p[i];        // 买得起的人数 × 单价
    if (sum >= ans2) { ans2 = sum; ans1 = p[i]; }   // 收入一样时取更低的价
}
```

> **要记得**：
> - 拆位前先 `int t = i`，别把循环变量自己改没了。
> - P2637 里价钱并列要取**更低**的那个价，所以写 `>=`，不是 `>`。
> - P1085 的 `ans` 初值给 1，别给 0。

二、模拟（06~10）

【模拟】题面讲什么就照着写什么，没什么技巧，考的是**别漏条件**。我的习惯是把题面的规则一条条抄成注释，再逐条写成 `if`。

题号	题目	我记的一句
06	P2670 扫雷	八个方向数雷，答案直接写回格子 `ans + '0'`；数组**外圈留一圈 0**，省掉越界判断
07	P1601 高精度加法	`reverse` 之后逐位加，`c` 存进位，最后别忘 `if (c) ans += '1'`
08	P2615 幻方	四条规则**按顺序**写成 if-else 链，第四条里还要判「右上方有没有被占」
09	P9198 轴对称	不要每次改完重扫整张图，维护一个「有多少对镜像格不相等」的数就行
10	P1098 字符串展开	先判「到底该不该展开」（两边同类且左 < 右），再按 `p1/p2/p3` 三层处理，否则原样输出 `-`

P9198 是这几题里最值钱的一个想法：**改一格，只更新这一格带来的变化。**

```
long long diff = 0;                  // 现在有多少对镜像格不相等
// 改之前：本来就不相等，先扣掉
if (j != m_j && !isEqual(img[i][j], img[i][m_j])) diff--;
img[i][j].rgb[t] = (img[i][j].rgb[t] + c) % 256;
// 改之后：变得不相等了，再加回来
if (j != m_j && !isEqual(img[i][j], img[i][m_j])) diff++;
cout << (diff == 0 ? "Yes\n" : "No\n");
```

> **要记得**：取模 256 要在加完之后做；中间那一列（`j == m-j+1`）自己跟自己对称，改它不影响结果，要跳过。

三、排序（11~15）

【排序】很多时候不是题目要你排序，是**排完序题目就变简单了。**原来要两两比较的，排完只用看相邻；原来要找的，排完可以二分。

题号	题目	我记的一句
11	P1177 排序	直接 `sort`
12	P1059 明明的随机数	`sort` 完再 `unique`，顺序不能反
13	P5143 攀爬者	按 `z` 排好，累加相邻两点的欧氏距离
14	P1104 生日	年、月、日一层层比，同生日时**后输入的排前面**（用 `id` 逆序）
15	B3784 演唱会	边读边累加每首歌的总欢乐度，排序取前 m，再处理「最喜欢的那首」

P1104 的多关键字比较，我写成一层层 `if`，最后拿 `id` 兜底：

```
bool cmp(const Stu& a, const Stu& b) {
    if (a.y != b.y) return a.y < b.y;      // 先比年
    if (a.m != b.m) return a.m < b.m;      // 再比月
    if (a.d != b.d) return a.d < b.d;      // 再比日
    return a.id > b.id;                    // 同一天生日：后输入的在前
}
```

> **要记得**：`sort(a, a+n)` 是左闭右开，别写成 `a+n+1`；`unique` 只去掉**相邻**的重复，所以必须先排序。`cmp` 里别写 `<=`，会破坏严格弱序。

四、贪心（16~20）

【贪心】每一步都拿眼前看起来最好的。难的不是写，是得能说清「为什么这么拿不会更差」——说不清就先别信，找个反例试试。

题号	题目	我记的一句
16	P2240 部分背包	按**单位价值** `v/m` 降序，能整块拿就整块拿，最后一块切开
17	P6832 子弦	答案就是**出现次数最多的那个单字符**的次数；任何子串出现 k 次，它的首字符至少也出现 k 次
18	P9517 drink	从第一个 1 到最后一个 1 的长度；一个 1 都没有就输出 0
19	P9045 橙汁	从左往右扫，遇到没见过的品牌就「提到前面」，代价是它前面还剩几个没固定的瓶子 `i-g-1`
20	P2676 书架	从高到低叠，累加到够高就停

P9045 里 `g` 记的是「已经固定在左边的瓶子数」，这个数想明白了代码就几行：

```
for (int i = 1; i <= n; i++)
    if (!used[a[i]]) {
        ans += i - g - 1;       // 中间隔着多少个「多余」的瓶子
        g++; used[a[i]] = true;
        if (g == k) { cout << ans; return 0; }
    }
cout << -1;
```

> **要记得**：贪心题先自己造个反例怼一下；怼不出来再写。P2240 别按总价值排，要按**单位价值**排。

五、前缀和 / 差分（21~24）

这一章的四道题其实是一家人：**把「对一整段做事」改成「只改两个端点」。**

题号	题目	我记的一句
21	P8218 区间和	先建 `s[i] = s[i-1] + x`，问 `[l,r]` 就答 `s[r] - s[l-1]`
22	P2367 语文成绩	区间 `[x,y]` 全体加 z：`d[x] += z; d[y+1] -= z`，最后再前缀和还原，边还原边取最小
23	P2004 领地选择	二维前缀和，套一个 C×C 的方框滑过去
24	P3397 地毯	二维差分的四个角 `+ - - +`，再求一次二维前缀和还原

一维的两段，几乎就是全部：

```
s[i] = s[i-1] + x;              // 前缀和：建表
cout << s[r] - s[l-1] << '\n';  // 问 [l,r] 的和

d[i] = a[i] - a[i-1];           // 差分：拿原数组直接造
d[x] += z;  d[y+1] -= z;        // 区间 [x,y] 全体 +z，只动两个端点
for (int i = 1; i <= n; i++) {
    a[i] = a[i-1] + d[i];       // 最后一次前缀和还原
    ans = min(ans, a[i]);
}
```

二维差分那四个角，符号我老是写反，抄一遍记牢：

```
d[x1][y1]++;     d[x1][y2+1]--;
d[x2+1][y1]--;   d[x2+1][y2+1]++;                    // 四角标记：+ - - +
d[i][j] += d[i-1][j] + d[i][j-1] - d[i-1][j-1];    // 还原
```

> **要记得**：
> - `s` 和 `d` 都开 `long long`；`1e5 × 1e4` 已经到 1e9 量级了。
> - `d[y+1]` 会用到 n+1，数组开到 `n+2`。
> - 二维那题答案**可能全是负数**，`maxSum` 初值给 `-1e9`，不能给 0。

六、二分（25~29）

【二分】这里其实是两件不同的事，别混：**在有序表里找位置**，和**猜答案再验证。**

题号	题目	我记的一句
25	P2249 查找	手写二分找「第一次出现」的位置，答案落在左半区
26	P1678 高考志愿	`lower_bound` 找到第一个不小于估分的位置，再跟**它前面那个**比谁更近
27	P1024 三次方程	逐米枚举小区间，两端异号就在里面二分逼近；`f(x)` 用秦九韶少算乘法
28	P1873 砍树	猜一个高度 H，看砍下来的木头够不够，够了就试着再高一点
29	P2440 木材加工	猜一段长度 len，看能切出几段

找左界和二分答案，骨架长得不一样，放一起对照最清楚：

```
// 找左界（P2249）：答案还能更左，就往左压
int l = 1, r = n;
while (l < r) {
    int mid = l + (r - l) / 2;
    if (a[mid] >= q) r = mid;
    else l = mid + 1;
}
if (a[l] == q) cout << l; else cout << -1;

// 二分答案（P1873 / P2440）：可行就试着再大一点
while (L <= R) {
    long long mid = L + (R - L) / 2;
    if (check(mid)) { ans = mid; L = mid + 1; }   // 可行 → 贪心往大走
    else R = mid - 1;
}
```

要记得：

- `mid` 写 `L + (R-L)/2`，别写 `(L+R)/2`，1e18 时后者会溢出。
- `check(mid)` 必须是**单调**的，不单调整道题就错了。
- `check` 里累加一超过需求就 `return true`，既提速又防溢出。
- P1678 记得处理 `pos == 0` 和 `pos == m` 这两种贴边情况。

七、双指针（30~34）

【双指针】两个指针都只往一个方向走，谁也不回头，于是省掉一层循环。看到「连续一段满足条件」就该想到它。

题号	题目	我记的一句
30	P1147 连续正整数和	和太小就把右端加进来，太大就把左端减掉
31	P7714 排列排序	`max == i` 的地方切块，块长大于 1 才要计费
32	P3143 钻石	先对每个前缀、后缀各求一次最优，再找个分割点把 `pre[i] + suf[i+1]` 拼起来
33	P1638 逛画展	右端一直往右吃，凑齐 m 位画家就试着缩左端，找最短区间
34	P3029 奶牛	同上，只是窗口的代价换成坐标差 `a[r].pos - a[l].pos`

P1638 的滑动窗口，答案**一定要写在里面那个 while 里**：

```
while (r < n) {
    r++; if (b[a[r]] == 0) cnt++;  b[a[r]]++;
    while (cnt == m) {                        // 条件满足，试着缩左边界
        if (r - l + 1 < ans) { ans = r-l+1; ansL = l; ansR = r; }
        b[a[l]]--; if (b[a[l]] == 0) cnt--;
        l++;
    }
}
```

> **要记得**：答案写在 `while (cnt == m)` **里面**才是「最短」，写在外面就变成「最后一个满足的窗口」了。P1638、P3029 都是这个坑。

八、递推（35~38）

【递推】小规模的结果算好了，大规模直接拿过来用。写之前先想清楚 `f[i]` 到底表示什么，这一句想明白，转移基本就出来了。

题号	题目	我记的 `f[i]`	怎么推
35	P1255 数楼梯	走到第 i 阶的方案数	`f[i] = f[i-1] + f[i-2]`，但要高精度
36	P1176 路径计数	走到 (i,j) 的方案数	`dp[i][j] = dp[i-1][j] + dp[i][j-1]`，有障碍就置 0
37	P1028 数的计算	以 i 开头的合法数列个数	`f[i] = f[i-1] + (i 是偶数 ? f[i/2] : 0)`
38	P1216 数字三角形	从这个点走到底能拿到的最大和	从下往上 `a[i][j] += max(a[i+1][j], a[i+1][j+1])`

P1216 我是从底往上写的，越往上路径越少，最后 `a[0][0]` 就是答案：

```
for (int i = r - 2; i >= 0; i--)
    for (int j = 0; j <= i; j++)
        a[i][j] += max(a[i+1][j], a[i+1][j+1]);
```

P1255 要单独说一句：N 能到 5000，斐波那契早就爆 `long long` 了，我是用**高精度 + 滚动数组**写的，只留 3 个 `vector`，用 `i % 3` 轮着用：

```
vector<int> f[3] = { {1}, {1}, {} };
for (int i = 2; i <= n; i++)
    f[i % 3] = add(f[(i-1) % 3], f[(i-2) % 3]);
```

> **要记得**：`f[1] = 1` 这种初值别漏（P1028）；取模要在**每一步**都做，别攒到最后（P1176 的 `% 100003`）。

九、搜索（39~44）

【搜索】先分岔：**要数「有多少种方案 / 有多少个连通块」用 DFS；要问「最少几步」用 BFS。**

题号	题目	我记的一句
39	P1706 全排列	`used` 数组记谁选过了，回溯三件套：终止、做选择、撤销选择
40	P1605 迷宫	四个方向数方案，到终点 `ans++` 后 `return`，走过的标记要**恢复**
41	P1596 水坑	八个方向灌水，进去就把格子改成 `.`，**不恢复**；主函数每遇到一个 `W` 就 `ans++`
42	P1746 离开中山路	BFS，`d` 数组既记步数又当 vis，入队时就标记
43	P1162 填涂颜色	反过来想：先从外圈 (0,0) 灌水标出「圈外的 0」，剩下没标到的 0 就是圈内，填 2
44	P1644 跳马	马只往右跳，四个固定方向 DFS 累计方案

回溯就这三步，写熟了所有「选/不选」的题都长这样：

```
void dfs(int step) {
    if (step == n + 1) { 输出; return; }      // 1. 到头了
    for (int i = 1; i <= n; i++)
        if (!used[i]) {
            used[i] = true;   ans[step] = i;  // 2. 做选择
            dfs(step + 1);
            used[i] = false;                  // 3. 撤销，恢复现场
        }
}
```

BFS 那边，标记要在**入队时**做，别等弹出：

```
queue<PII> q;  q.push({x1, y1});  d[x1][y1] = 0;   // d 兼作 vis
while (!q.empty()) {
    auto t = q.front(); q.pop();
    int x = t.first, y = t.second;
    if (x == x2 && y == y2) return d[x][y];
    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i], ny = y + dy[i];
        if (合法 && d[nx][ny] == -1 && m[nx][ny] == '0') {
            d[nx][ny] = d[x][y] + 1;               // 入队即标记
            q.push({nx, ny});
        }
    }
}
```

要记得：

- 数方案的 DFS 忘了恢复现场，答案会偏小；数连通块的灌水**不能**恢复，否则重复数。
- BFS 等弹出才标记，同一格会重复入队，内存直接爆。
- 网格题数组四周留一圈 0，越界判断全省了（P1162 尤其关键）。
- `memset(d, -1, sizeof d)` 一步把「没访问过」初始化好。

十、离散化（45~46）

【离散化】坐标能到 1e9，数组开不了那么大；但真正出现的数只有 n 个。那就**不管数本身多大，只记它排第几。**

三步：收集所有要用的值 → 排序去重 → 用 `lower_bound` 查它排第几。

```
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());   // 排序 → 去重
int r = lower_bound(vals.begin(), vals.end(), x) - vals.begin() + 1;  // 映射成 1..m
```

题号	题目	我记的一句
45	P1496 火烧赤壁	端点差分：`diff[u]++`、`diff[v]--`，扫的时候 `cur>0` 就把「基本区间」`vals[i+1]-vals[i]` 加进答案
46	P1908 逆序对	离散化之后上【树状数组】，从右往左扫，每读一个数先问「右边已经出现、且比我小的有几个」

P1908 我拆开讲过，因为【树状数组】确实不太好懂。关于它推荐这个视频，我当初也是先看视频才直观起来的：

> https://www.bilibili.com/video/BV1ce411u7qP/
> 另外 `本章手写笔记/` 里的【数状数组笔记.pdf】和【洛谷P1908 逆序对(离散化).pdf】也建议先翻一遍（不要求第一遍就看懂）。

`lowbit` 举个小例子：下标 1~8 的 `lowbit` 依次是 `1, 2, 1, 4, 1, 2, 1, 8`。分层看，等于 1 的（1,3,5,7）在最底层只管自己，等于 2 的（2,6）各管两个，等于 4 的（4）管四个，等于 8 的（8）管八个——它就是这样形成一棵看不见的树的。

```
int lowbit(int x) { return x & -x; }                        // 取出二进制最低位的 1
void add(int i, int v) { while (i <= n) { bit[i] += v; i += lowbit(i); } }   // 往上爬
int sum(int i) { int s = 0; while (i > 0) { s += bit[i]; i -= lowbit(i); } return s; } // 往回剥

for (int i = a.size() - 1; i >= 0; i--) {                   // 从右往左扫
    int rank = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
    ans += sum(rank - 1);       // 右边已经出现、且比我小的个数
    add(rank, 1);
}
```

> **要记得**：
> - **倒着扫**是为了让 `sum` 统计到的都是「右边」的数，天然满足 `i < j`，不用再判一次。
> - `bit` 开**去重后的 m** 那么大就够，不是原来的 n。
> - `ans` 用 `long long`。
> - P1496 题目是**左闭右开**，所以累加的是 `vals[i+1] - vals[i]` 这种「基本区间」的长度，坐标也要 `long long`。

十一、回头看，其实就三条线

把十章合起来看，来来回回就这几件事：

1. **好多题是「排完序就好写了」。** P2637、P5143、P2676 排完只用看相邻；P2249、P1678、P1873、P2440 排完才能二分；P1059、P1908 排完才能 `unique`；P3029、P3143 排完才能滑窗口。卡住的时候先问自己一句：**排个序会不会变简单？**
2. **别每次都从头算一遍。** P9198 的 `diff`、P2367 的差分、P1638 的 `cnt`、P3029 的 `map` 计数，都是「只更新被改动的那部分」。凡是题目说「每次操作后问整体状态」，就往这边想。
3. **区间那几道题是一家人。** 多次问区间和用前缀和，多次区间加用差分，找连续一段用双指针，找最小/最大可行值用二分答案，值域太大就先离散化。换的只是「先改还是先问」的顺序。

十二、要记得（提交前再过一遍）

1. `int` 溢出：前缀和、砍树累加、逆序对答案、坐标差、背包价值，看到就直接 `long long`。
2. `d[y+1]` 越界：区间加的右端点要 `+1`，数组开 `n+2`。
3. `unique` 之前没排序：它只去相邻重复。
4. 二分边界：找左界是 `l<r` + `r=mid`，二分答案是 `l<=r` + 拿 `ans` 记，别混。
5. 滑动窗口的答案写在里面那个 `while` 里，才是最短。
6. DFS 忘了恢复现场 / BFS 忘了入队即标记：一个答案偏小，一个重复入队。
7. 0 基 1 基混用：读进来是 1~n 就全程 1 基；P2004 转置存了，输出要反着来。
8. 边界没测：`n=1`、全是 0（P9517 输出 0）、切不出 1cm（P2440 输出 0）、答案全负（P2004 初值 `-1e9`）。

> 顺手记两个我自己代码里还能改的地方（**没动原文件**，先记着）：
>
> - **P1085**：`ans` 初值为 0，且 `max` 从第 1 天开始取。要是只有第 1 天超过 8 小时，后面没有更优的，就会输出 0（应该是 1）。样例恰好第 3 天最大，所以没暴露。把 `ans` 初值给 1、或让 `max` 从 -1 起就行。
> - **P1746**：`bfs()` 末尾没写 `return`。题目保证有解时走不到，但严格说是未定义行为，补个 `return -1;` 更稳。

十三、46 题速查表

#	题目	章	一句话
01	P1179 数字统计	枚举	逐个数拆位统计 2
02	P2637 拍卖	枚举	降序排序，枚举成交价 × 人数
03	P1085 不高兴的津津	枚举	打擂台找最大 `a+b`
04	P6437 JACK	枚举	三层枚举取 ≤m 的最大和
05	P8834 序列	枚举	二重枚举比乘积
06	P2670 扫雷	模拟	八方向数雷，外圈留 0
07	P1601 高精度加法	模拟	反转后逐位加、进位
08	P2615 幻方	模拟	四条规则写成 if-else 链
09	P9198 轴对称	模拟	维护不相等的镜像对数
10	P1098 字符串展开	模拟	判同类 → 按 p1/p2/p3 填充
11	P1177 排序	排序	`sort`
12	P1059 明明的随机数	排序	`sort` + `unique`
13	P5143 攀爬者	排序	按 z 排序累加距离
14	P1104 生日	排序	多关键字 cmp + id 逆序

#	题目	章	一句话
15	B3784 演唱会	排序	累加总和 → 排序 → 调最爱
16	P2240 部分背包	贪心	单位价值降序，最后一块切分
17	P6832 子弦	贪心	答案 = 出现最多的单字符次数
18	P9517 drink	贪心	第一个 1 到最后一个 1 的长度
19	P9045 橙汁	贪心	新品牌前移，代价 `i-g-1`
20	P2676 书架	贪心	从高到低叠到够高
21	P8218 区间和	前缀和	`s[r]-s[l-1]`
22	P2367 语文成绩	差分	区间加 → 前缀和还原取 min
23	P2004 领地选择	前缀和	C×C 方框滑窗
24	P3397 地毯	差分	四角 `+ - - +` 再还原
25	P2249 查找	二分	手写左界二分
26	P1678 高考志愿	二分	`lower_bound` 后比左右邻居
27	P1024 三次方程	二分	逐米判号 + 区间二分
28	P1873 砍树	二分	`check(H)` 猜高度
29	P2440 木材加工	二分	`check(len)` 数段数
30	P1147 连续和	双指针	小了加 r、大了减 l
31	P7714 排列排序	双指针	`max==i` 处切块，块长>1 计费
32	P3143 钻石	双指针	分割点拼 `pre[i]+suf[i+1]`
33	P1638 逛画展	双指针	覆盖 m 位画家的最短区间
34	P3029 奶牛	双指针	覆盖全部品种的最小坐标差
35	P1255 数楼梯	递推	斐波那契 + 高精度滚动
36	P1176 路径计数	递推	上 + 左，障碍置 0，取模
37	P1028 数的计算	递推	`f[i]=f[i-1]+(偶? f[i/2]:0)`
38	P1216 数字三角形	递推	自底向上取 max
39	P1706 全排列	搜索	`used` 数组回溯
40	P1605 迷宫	搜索	四方向计数 + 恢复现场
41	P1596 水坑	搜索	八方向染色计数
42	P1746 离开中山路	搜索	BFS，`d` 兼 vis
43	P1162 填涂颜色	搜索	从外圈灌水，剩下的 0 填 2
44	P1644 跳马	搜索	四个固定方向累计方案
45	P1496 火烧赤壁	离散化	端点差分 + 基本区间长度

#	题目	章	一句话
46	P1908 逆序对	离散化	倒序 `sum(rank-1)` 再 `add`

十四、接下来

46 题停在【离散化】+【树状数组】这个门槛上，往下的路挺清楚的：

1. **树状数组再进一步**：区间改 + 单点查、区间改 + 区间查、二维树状数组。
2. **线段树**：区间最值、懒标记。逆序对用线段树也能写，正好跟 P1908 对拍。
3. **DP 系统化**：01 背包、完全背包、区间 DP、LIS 的 O(n log n) 写法（P3143 已经碰到 LIS 的意思了）。
4. **图论**：邻接表、最短路、拓扑排序、并查集。
5. **字符串**：哈希、KMP、Trie。

复习的话，按章回做就行，每章挑 1 题**关掉笔记自己重写一遍**；隔一周、隔一个月再来一轮。能一次写对，才算真的会了。